

Software Requirements and Design Document For Group 2

Version 3.0

Authors:

Colin Maloney

Alex Nino

Kobus VanSteenburg

Samuel Pauly

Zachary Kupersmith

1. Overview (5 points)

Track Tracker is a music streaming data analytics tool. The tool ingests trending sounds from TikTok and corroborates data from Spotify to identify tracks with the potential to become future hits. We use the TikTok-API (unofficial) and Spotify Web API for data collection, with web-scraping to supplement stream counts in place of the deprecated popularity score. For backend services and data ingestion we use Python with FastAPI, and for front-end we use Next.JS with TypeScript. All data is stored in PostgreSQL on AWS RDS. In Increment 3, the platform was extended with a full Charts page, dark mode, enhanced track modals with analytics, album artwork, a contact form, an About page, and accurate library-wide statistics.

2. Functional Requirements (10 points)

1. **High** – The system shall search TikTok for videos across a configurable list of music-discovery hashtags (e.g., #newmusic, #indiemusic, #underratedartist).
2. **High** – The system shall extract sound title and author username from each TikTok video, filtering out original or untitled sounds.
3. **High** – The system shall deduplicate discovered TikTok sounds by their TikTok sound ID before passing them to Spotify enrichment.
4. **High** – The system shall search the Spotify Web API using the extracted artist and track names.
5. **High** – The system shall persist all track, artist, and enrichment data in a PostgreSQL database.
6. **High** – The system shall prevent duplicate tracks and artists by enforcing uniqueness using the Spotify ID.
7. **High** – The system shall record a TrackSnapshot on every ingestion run capturing Spotify stream count (via web scraping) and a timestamp for time-series momentum tracking.
8. **High** – The system shall expose a Charts page displaying all tracked songs with search and sorting functionality.
9. **High** – The system shall display album artwork on track cards across the home and charts pages.
10. **High** – The system shall display an expandable modal for each track showing detailed analytics including a stream-over-time line graph.
11. **High** – The system shall display accurate library-wide statistics (total tracks tracked, total streams) on the home page using a dedicated `get_library_stats` query.
12. **High** – The system shall support track ranking within the track list based on stream momentum.
13. **Medium** – The system shall expose a REST API via FastAPI with OAuth2 token-based authentication.

14. **Medium** – The system shall support full dark mode across all pages and components.
15. **Medium** – The system shall log all ingestion and enrichment failures.

3. Non-functional Requirements (10 points)

1. **High** – The system shall handle failed API calls gracefully by catching exceptions and continuing pipeline execution without crashing.
2. **High** – The system shall use database connection pool pre-ping to recover from stale connections after PostgreSQL restarts.
3. **High** – The system shall use a composite index on TrackSnapshot (track_id, recorded_at) to optimize time-series queries.
4. **Medium** – The system shall provide structured logging for all pipeline stages.
5. **Medium** – The system shall use Alembic for versioned database schema migrations.
6. **Medium** – The system shall use environment variables for all secrets and connection strings with no hardcoded credentials in the codebase.

4. Use Case Diagram (10 points)



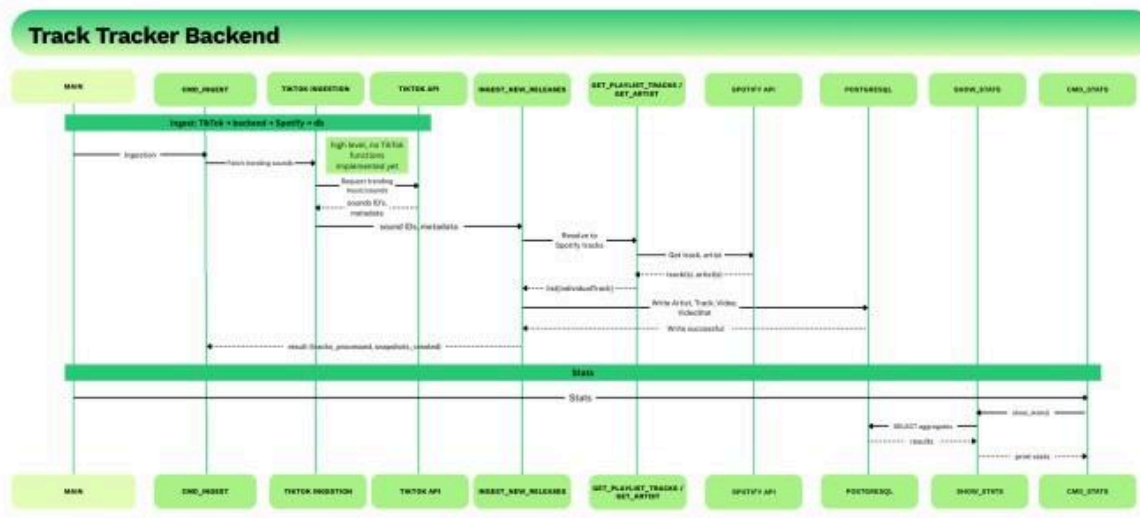
Textual Descriptions:

View Top 10 Growing Tracks: When a user visits the website, the landing page displays the top 10 growing tracks along with accurate library-wide statistics (total tracks tracked and total streams). Album artwork is shown on each track card. No preconditions or postconditions.

Search for Songs: Users can type into the search bar on the Charts page to search the full database of tracked songs. The Charts page supports sorting and displays all ingested tracks. No preconditions or postconditions.

View Song Details / Analytics: Users can click any track card to open an expanded modal showing detailed song analytics, including a line graph of stream counts over time pulled from stored snapshots..

5. Class Diagram and/or Sequence Diagrams (15 points)



6. Operating Environment (5 points)

The Track Tracker system operates across two environments:

- Backend: Python 3.11+ running on any POSIX-compatible OS (Linux/macOS). Deployed as a FastAPI service. Requires network access to the Spotify Web API, TikTok (unofficial) API, and a PostgreSQL instance on AWS RDS.
- Frontend: Next.JS 14 with TypeScript, deployed on Vercel. Requires a modern web browser (Chrome, Firefox, Safari, Edge). Styled with Tailwind CSS; supports both light and dark mode.
- Database: PostgreSQL 15 hosted on AWS RDS. Schema managed with Alembic migrations. Credentials are provided exclusively via environment variables; no secrets are hardcoded.

7. Assumptions and Dependencies (5 points)

Dependencies:

- **Unofficial TikTok API:** Third-party API used for TikTok data ingestion. Dependent on the API remaining functional and up to date.

- **Spotify Web API:** Used for track and artist metadata enrichment. Subject to Spotify's rate limits and terms of service.
- **Spotify Stream Scraping:** Stream counts are obtained via web scraping as the popularity score field is deprecated. Dependent on Spotify's web page structure remaining consistent.
- **AWS RDS (PostgreSQL):** Managed database service. The system assumes continuous availability of the RDS instance and valid connection credentials via environment variables.